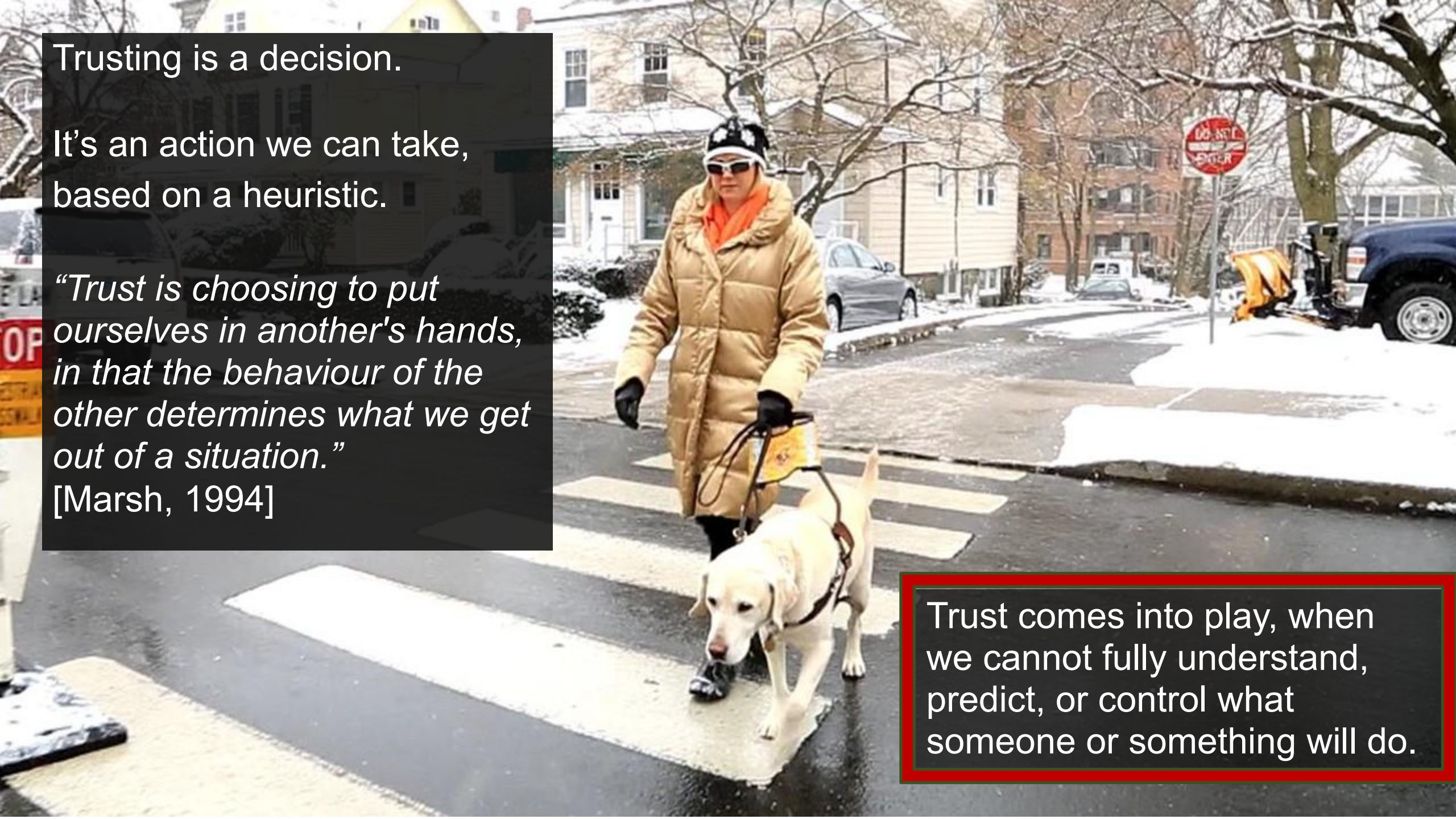
OntarioTech UNIVERSITY

Black Boxes – Where Do They Come From?

First, a little recap to set context…



Trusting is a decision.

It’s an action we can take, based on a heuristic.

“Trust is choosing to put ourselves in another's hands, in that the behaviour of the other determines what we get out of a situation.”
[Marsh, 1994]

Trust comes into play, when we cannot fully understand, predict, or control what someone or something will do.

Trustworthiness Features as Relevant to machines

What information is useful?

Trustworthiness integrates black-box and white-box information sources.

Competence	*Can this machine do what I need it to?* *Are the machine's* ***capabilities*** sufficient to *pursue its goals?* Testing, verification, patching, explanations, **interpretability**.
Predictability & Meta-Predictability	*How predictable or reliable is this machine's behaviour?* **Statistics**, based on observed behaviour. Reputation systems.
Honesty & Integrity	Ability to make and stick to **agreements**. *Can the machine revise its behaviour as a result of an interaction with me?* *Do I understand when can this change, based on new information?*
Willingness & Benevolence	*What is the machine 'trying' to do?* What are its **goals**? Are these explicitly captured in the machine? Are the machine's goals **available** to and **interpretable** by me? What are the **implicit goals** captured in the machine's design?

How interpretable are today's AI systems?

Let's consider Neural Networks…

Black Boxes?

Why do we tend to consider neural networks to be black boxes?

Is it just really hard mathematics?

Or is there something that makes them fundamentally not understandable?

What do Neurons *do?*

Each neuron (in a deep neural network)

- receives a set of inputs from other neurons,
- multiplies each by a weight it learned during training,
- and then computes an output (by passing it through a nonlinear activation function).

This is mathematically equivalent to drawing a linear decision boundary
in the space of inputs.

E.g., a line in 2 dimensions or a hyperplane in higher dimensions.

Let’s explore an example…

Example: Logic with Simple Neurons

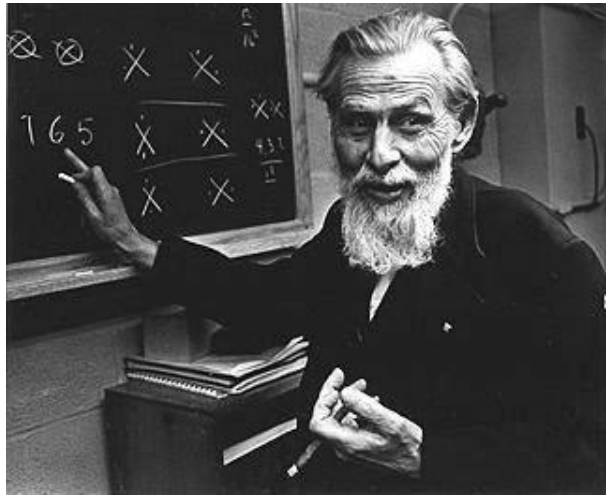
The McCulloch-Pitts (MCP) Neuron Model

A highly simplified model of a neuron.

A great example of interdisciplinary research:

McCulloch was a neuroscientist;

Pitts was a logician.



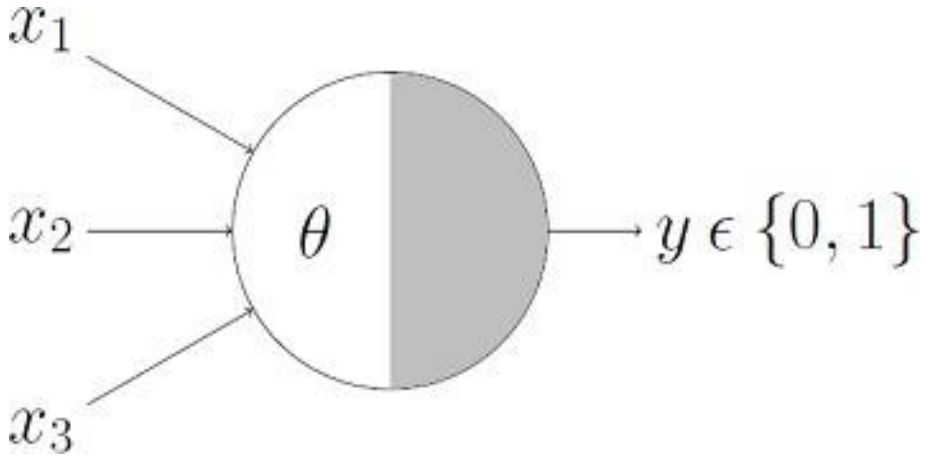
The McCulloch-Pitts (MCP) Neuron Model

- The MCP Neuron has a binary output $y \in \{0, 1\}$
- $y = 1$ indicates that the neuron ***fires*** and $y = 0$ that it is at rest.
- It has a number N of ***exitatory*** binary inputs $x_k \in \{0, 1\}$.
- It has a single ***inhibitory*** input i. If i has been activated, the neuron cannot fire.
- It has a ***threshold*** value Θ. If the sum of its inputs is larger than Θ, the neuron fires. Otherwise, it stays at rest.

Given the input $x = [x_1, x_2, x_3, \ldots, x_n]$, the inhibitory input i and the threshold Θ, the output y is computed as:

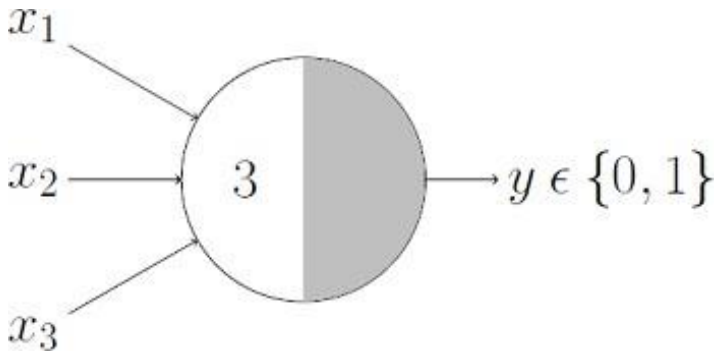
$$\sigma(\mathbf{x}) = \begin{cases} 1 \text{ if } \sum_{k=1}^{n} x_k > \Theta \text{ and } i = 0, \\ 0 \text{ otherwise.} \end{cases}$$

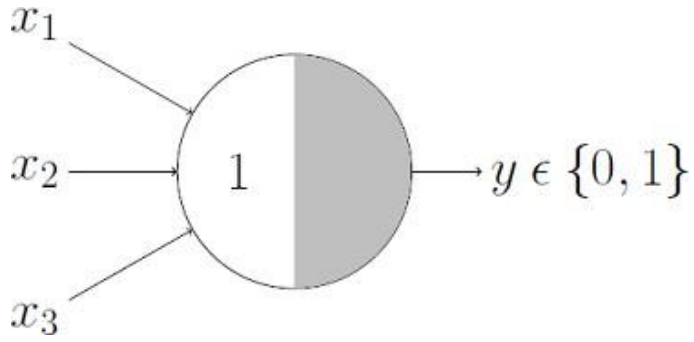
Logic with McCulloch-Pitts Neurons

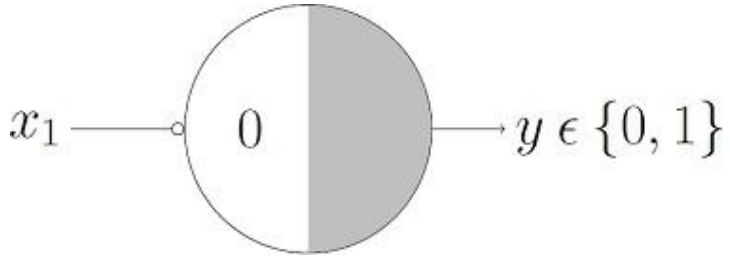


Recall that inputs and outputs are ***single binary digits***.

Our job is to specify the ***threshold value***.

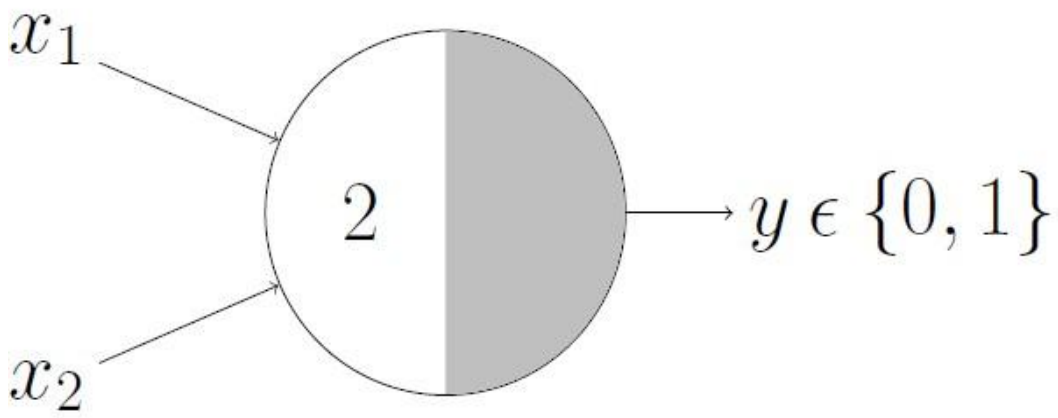






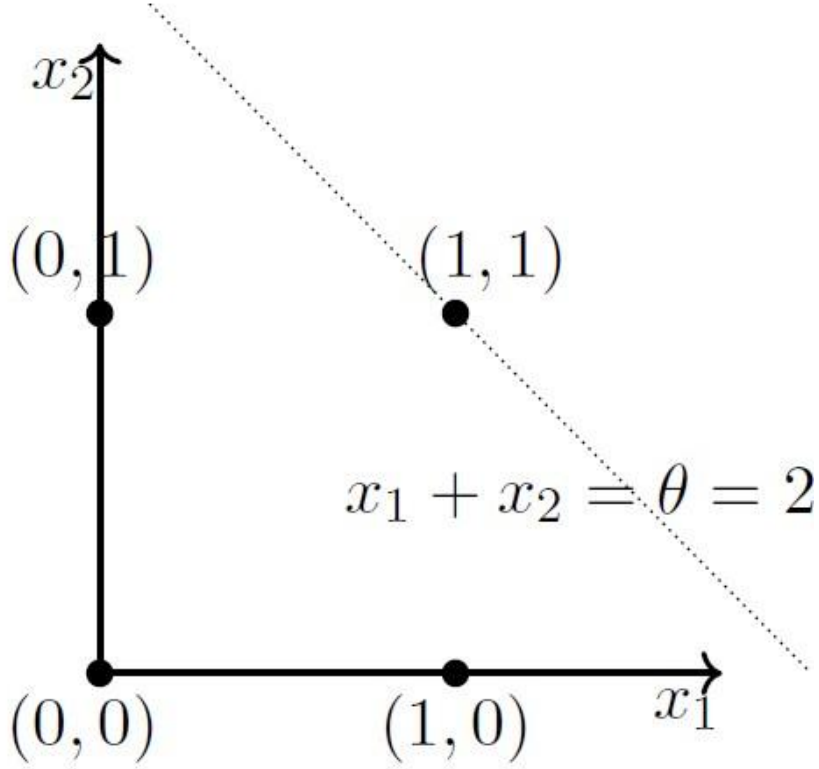
What Boolean logic functions can we capture using a single MCP Neuron?

Geometric Interpretation of AND in MCP Neurons

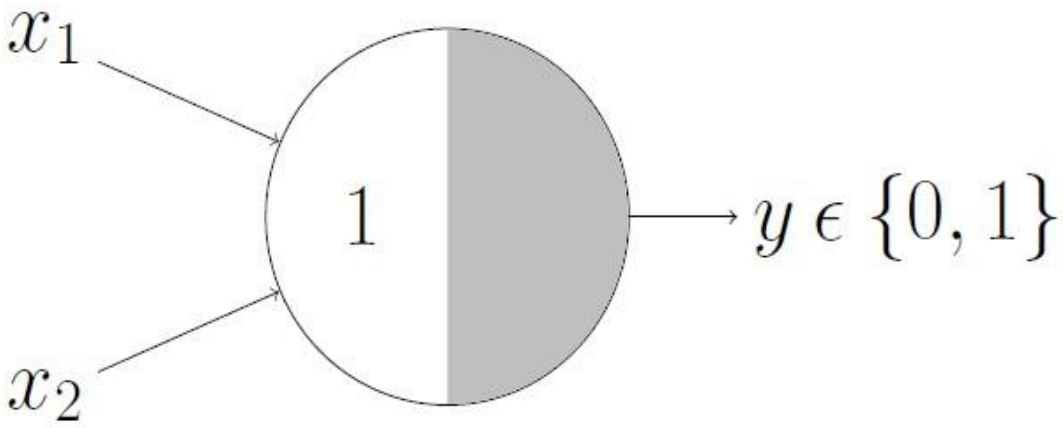


AND function

$$x_1 + x_2 = \sum_{i=1}^{2} x_i \geq 2$$

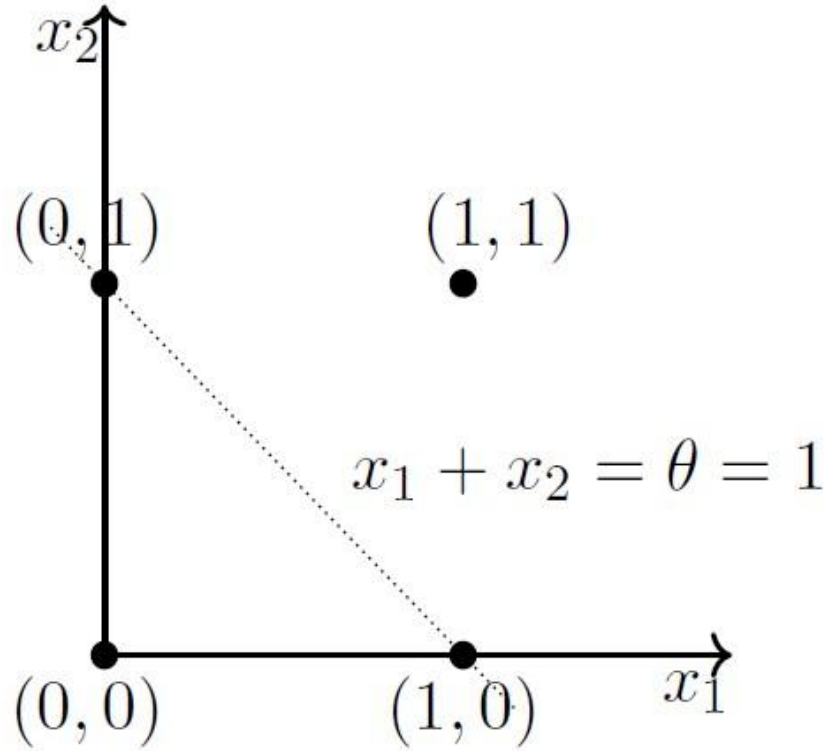


Geometric Interpretation of OR in MCP Neurons

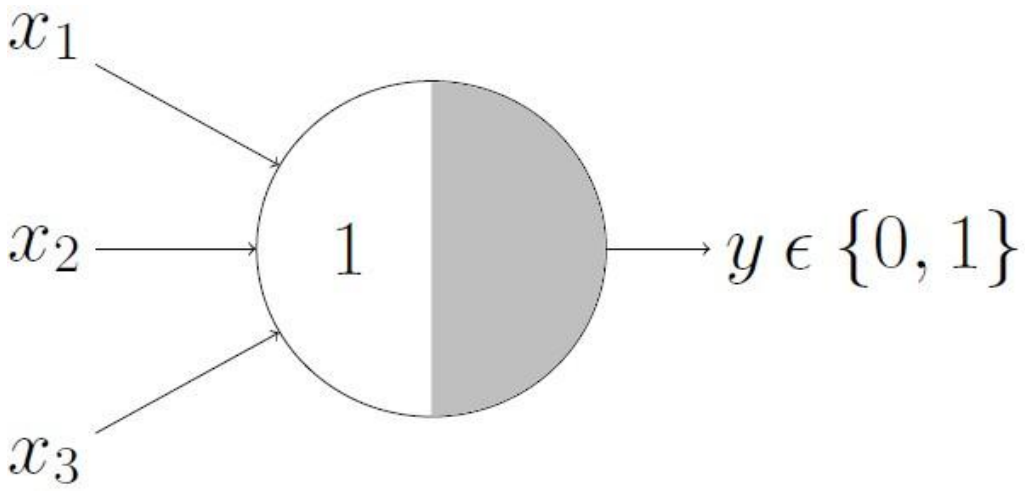


OR function

$$x_1 + x_2 = \sum_{i=1}^{2} x_i \geq 1$$

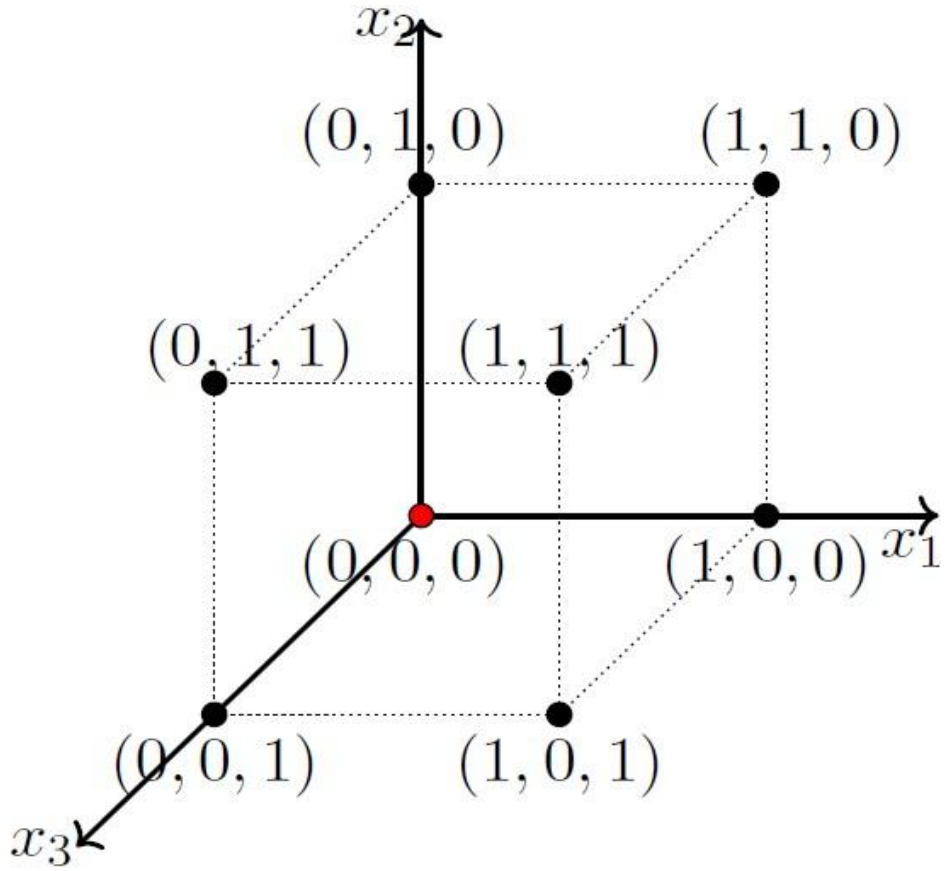


Geometric Interpretation of OR with 3 Inputs

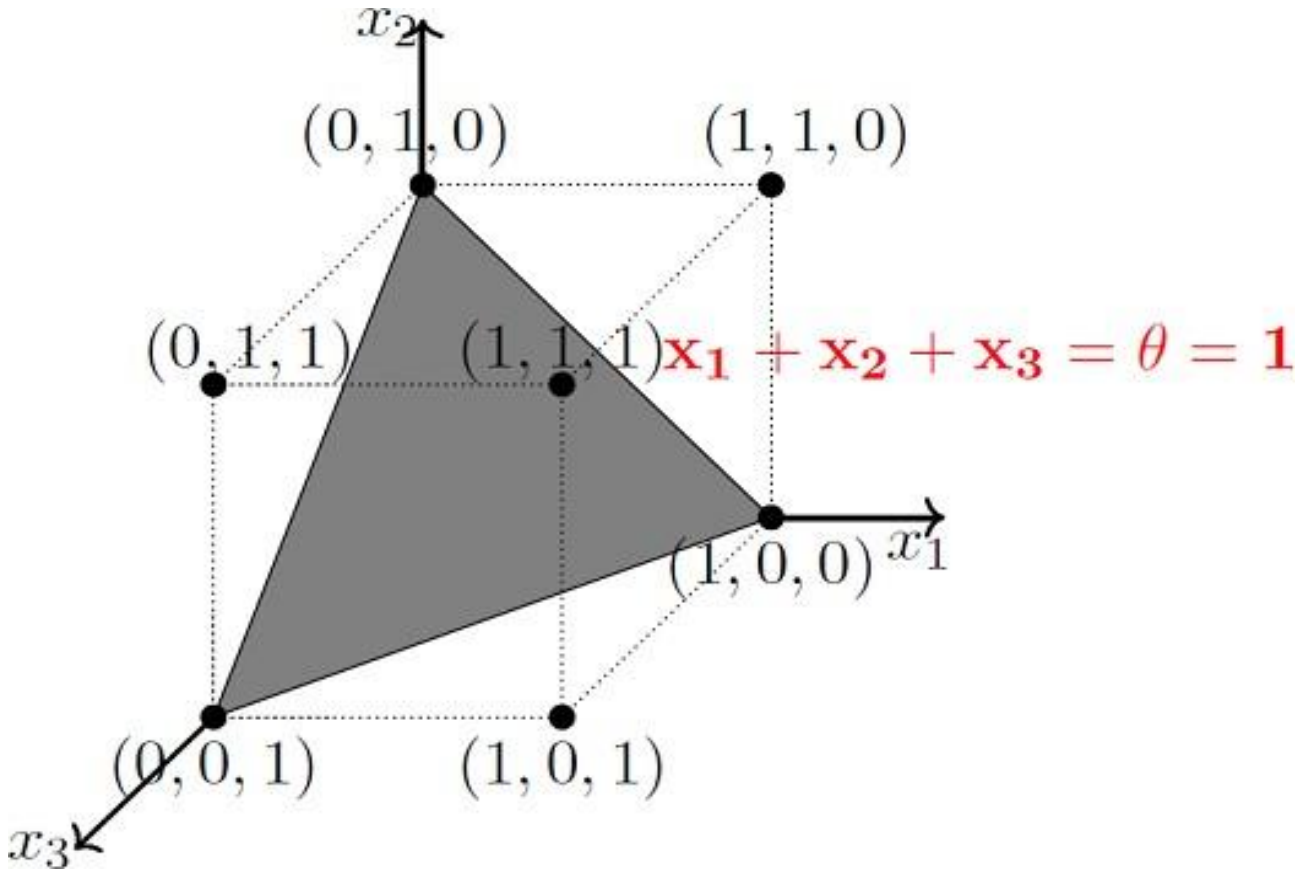


OR function

$$x_1 + x_2 + x_3 = \sum_{i=1}^{3} x_i \geq 1$$



Geometric Interpretation of MCP Neurons



Linear separability
(for Boolean functions):

There exists a line (plane)

such that all inputs which produce a 1 lie on one side of the line (plane)

and all inputs which produce a 0 lie on other side of the line (plane).

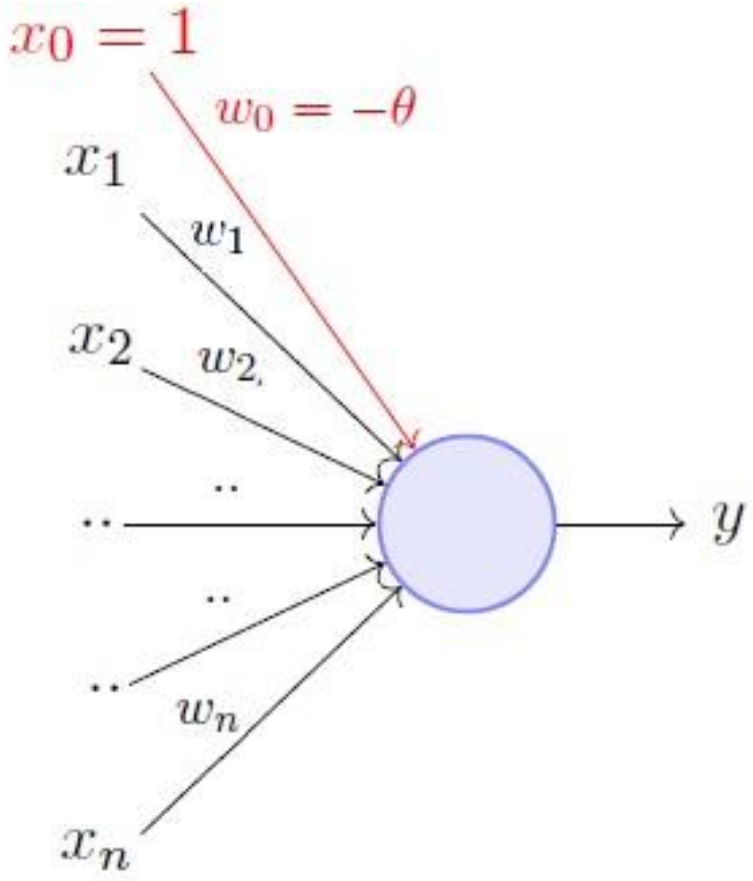
Check your understanding: XOR is not linearly separable. Can you demonstrate this geometrically?

Limitations of MCP Neurons

- *What about non-Boolean (say, real valued) inputs?*
- *Do we always need to hand-code the threshold?*
- *Should all inputs always be treated equally?*
  What if we want to assign more importance to some inputs?
- *What about functions which are not linearly separable?*
  (E.g., the XOR function)

Rosenblatt’s Perceptron

Rosenblatt's Perceptron



$$y = 1 \quad if \sum_{i=0}^{n} w_i * x_i \geq 0$$

$$= 0 \quad if \sum_{i=0}^{n} w_i * x_i < 0$$

$$where, \quad x_0 = 1 \quad and \quad w_0 = -\theta$$

Perceptrons vs MCP neurons:

- Numerical weights (a measure of importance) for inputs,
- Supports real valued inputs - which makes it more useful and generalized.

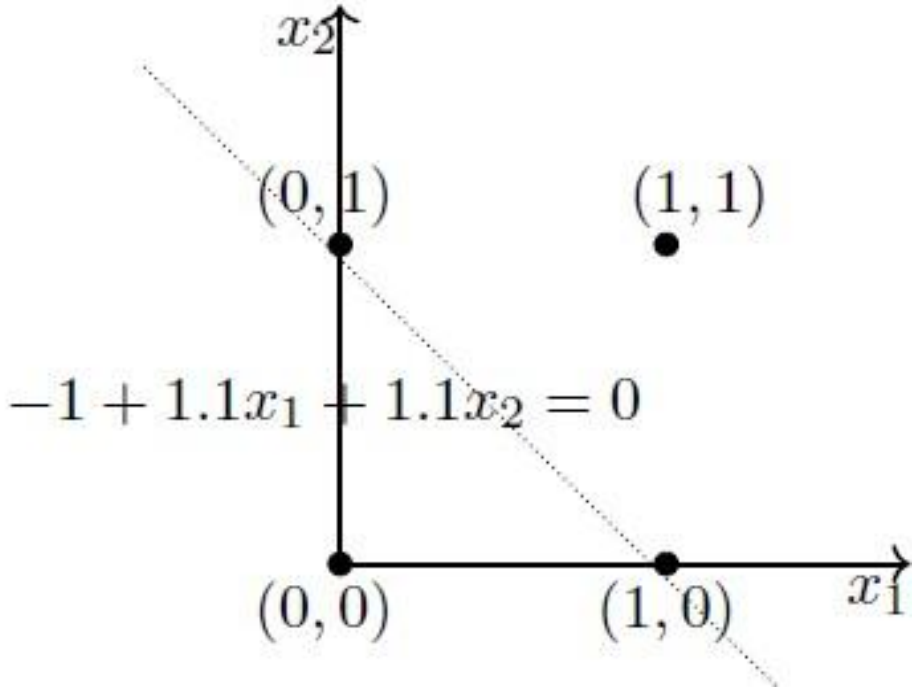
Logic with Perceptrons: OR Example

x_1	x_2	OR	
0	0	0	$w_0 + \sum_{i=1}^{2} w_i x_i < 0$
1	0	1	$w_0 + \sum_{i=1}^{2} w_i x_i \geq 0$
0	1	1	$w_0 + \sum_{i=1}^{2} w_i x_i \geq 0$
1	1	1	$w_0 + \sum_{i=1}^{2} w_i x_i \geq 0$

$$w_0 + w_1 \cdot 0 + w_2 \cdot 0 < 0 \implies w_0 < 0$$
$$w_0 + w_1 \cdot 0 + w_2 \cdot 1 \geq 0 \implies w_2 > -w_0$$
$$w_0 + w_1 \cdot 1 + w_2 \cdot 0 \geq 0 \implies w_1 > -w_0$$
$$w_0 + w_1 \cdot 1 + w_2 \cdot 1 \geq 0 \implies w_1 + w_2 > -w_0$$

One possible solution is

$w_0 = -1,\ w_1 = 1.1,\ w_2 = 1.1$



Check your understanding: How would the logic for AND & NOT gates look?

A Quick Aside…

Robust Logic with Perceptrons

What would happen if the input ***signals degraded*** somewhat?
What if the weights or neural hardware was not perfect?

Consider the case of noisy inputs.

x_1	x_2	x_1 AND x_2
0.2	-0.1	0
0.1	1.05	0
1.1	1.3	0
0.91	1.0	1

Neural computing is often considered to be ***robust***:

It can still often give us the right answer, with a small amount of degradation.

What do Neural *Networks* do?

A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

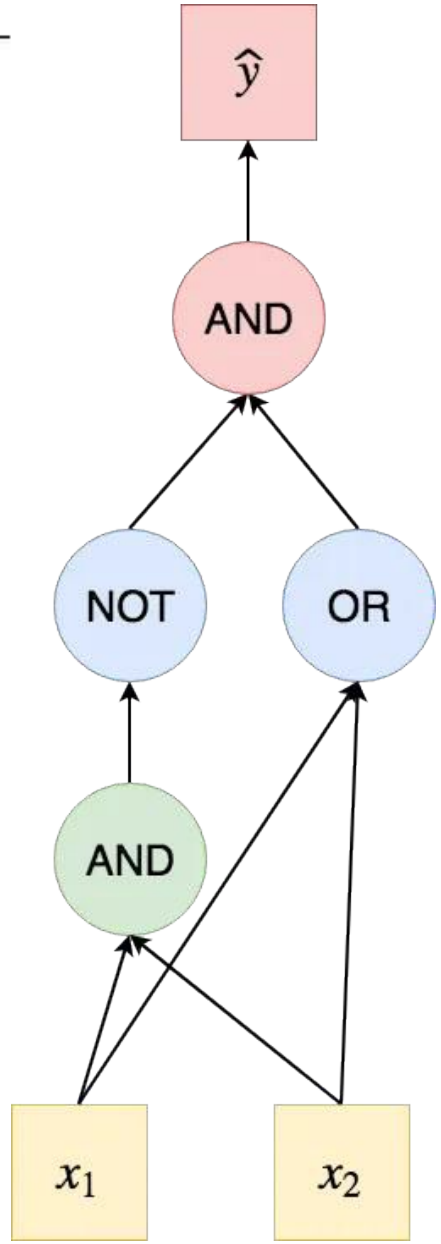
So, what about that pesky XOR problem?

If we want to write down the function for XOR using logic, we must combine multiple ‘fundamental’ operators:

```
XOR(A,B) = AND(NOT(AND(A,B)),OR(A,B))
```

We can implement this logic in a neural network by ‘chaining’ neurons together. This requires a new approach to training since the network will no longer be linearly separable, such as:

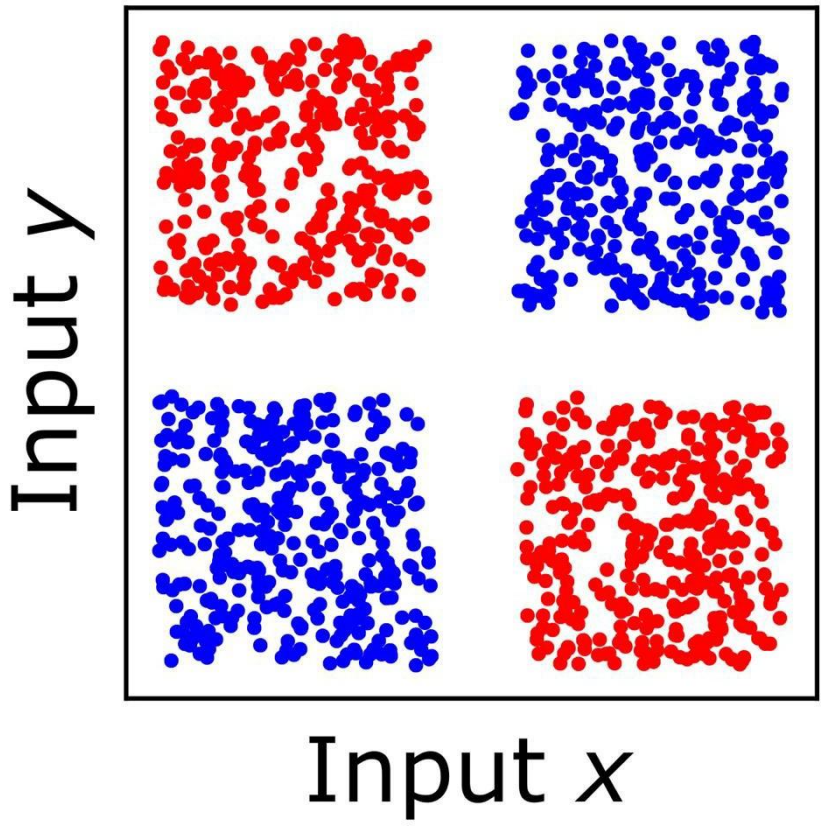
- Backpropagation!
- Evolution(ary algorithms)



The Black Box Problem Arises As We Consider How Multiple Layers of Neurons Interact in a Neural Network

Example: Simple Classification

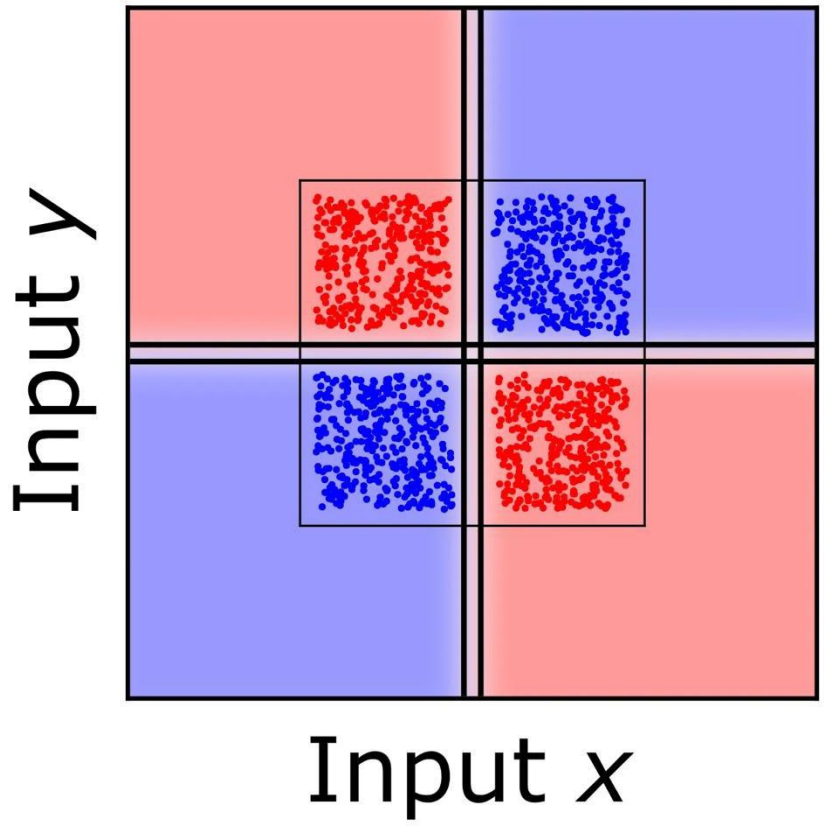
Consider this classification problem…



Based on an example by Paul J. Blazek:
https://towardsdatascience.com/why-we-will-never-open-deep-learnings-black-box-4c27cd335118

Example: Simple Classification

We might expect that a machine learning model would learn a classifier like this…

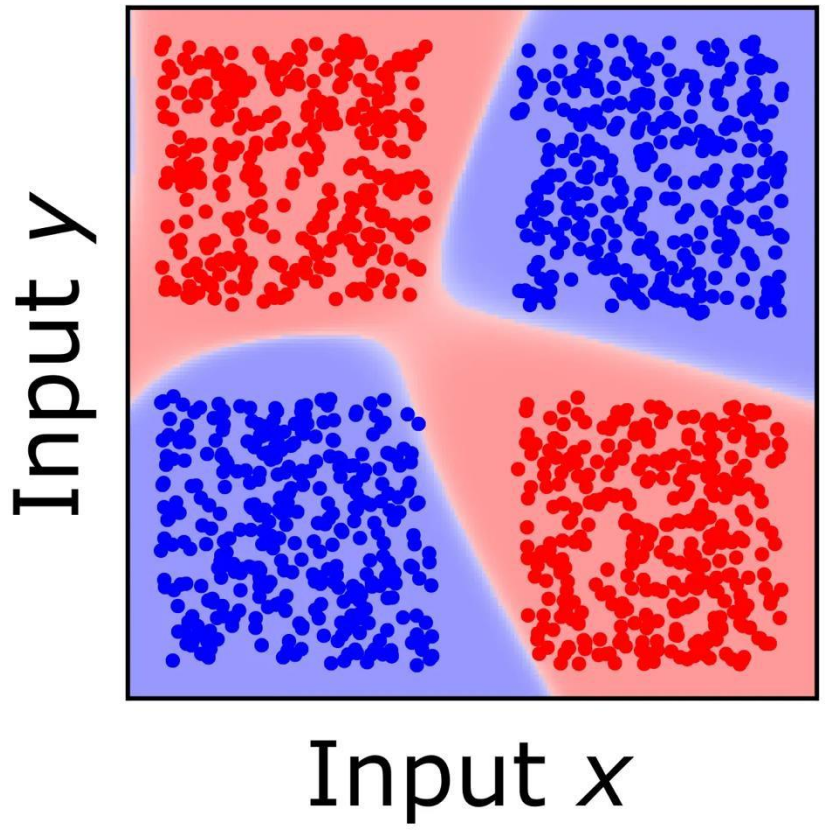


Based on an example by Paul J. Blazek:
https://towardsdatascience.com/why-we-will-never-open-deep-learnings-black-box-4c27cd335118

Example: Simple Classification

But when we train (using backpropagation) a Neural Network with 2 layers each containing 4 neurons, we instead get decision boundaries like this…

Why?



Based on an example by Paul J. Blazek:
https://towardsdatascience.com/why-we-will-never-open-deep-learnings-black-box-4c27cd335118

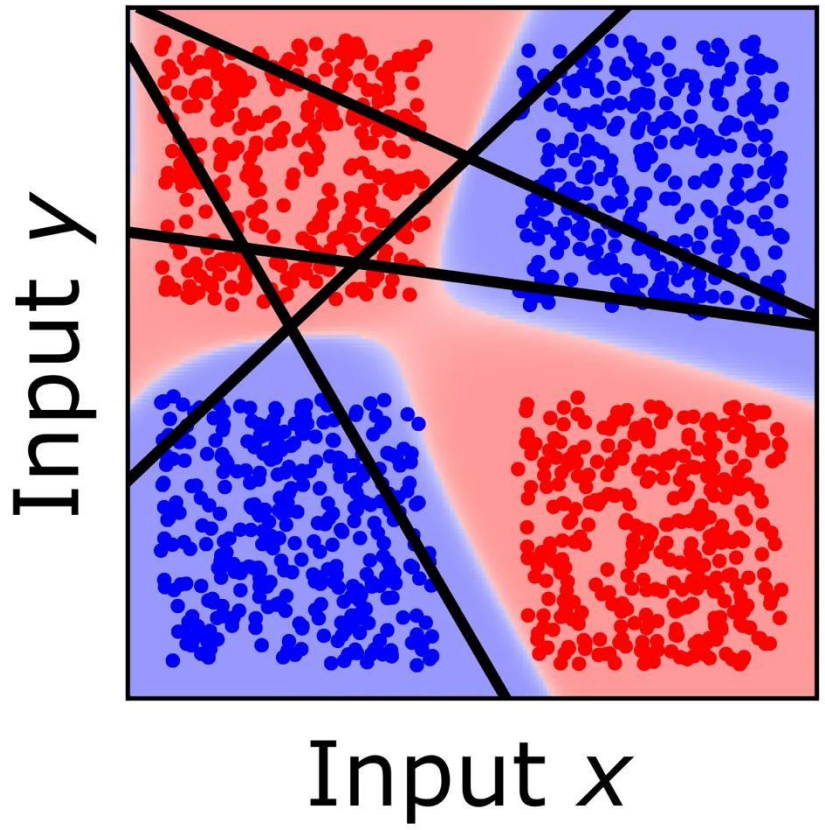
Example: Simple Classification

It's to do with how it is trained.

Let's look at what the ***first layer*** of neurons learns to model…

What do these lines represent? They don't seem to make much sense, so why did the network learn them?

This is the beginnings of what we mean when we say that Neural Networks can be 'black box'.



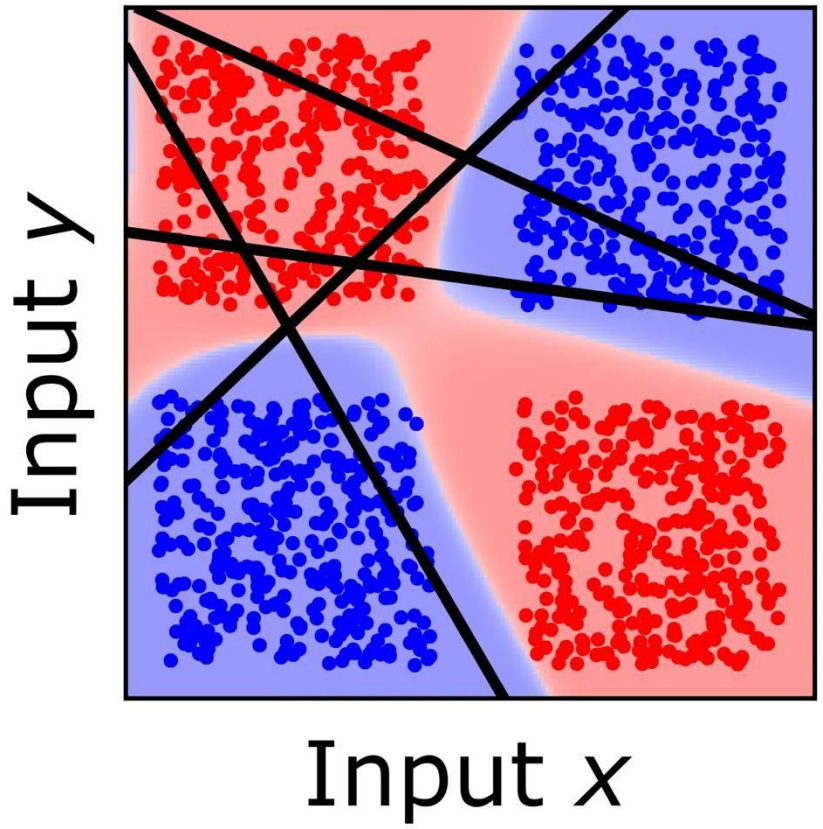
Based on an example by Paul J. Blazek:
https://towardsdatascience.com/why-we-will-never-open-deep-learnings-black-box-4c27cd335118

Example: Simple Classification

Ok, so we know from our logic gates example that we can't get it right with one layer, so this might be a reasonable starting point.

We can think about the first layer being the **'building blocks'** for the later layers, since they are the second layer's input.

So why ***these*** building blocks?



Based on an example by Paul J. Blazek:
https://towardsdatascience.com/why-we-will-never-open-deep-learnings-black-box-4c27cd335118

Example: Simple Classification

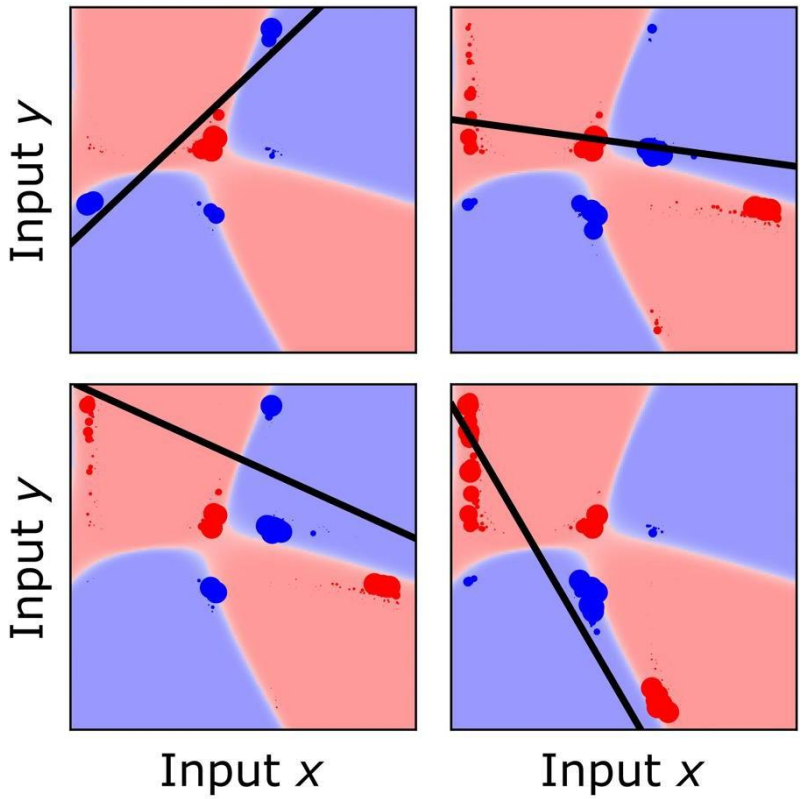
Let's examine where they came from.

In this figure, we see each first layer neuron separately…

…with the training samples shown with size proportional to ***how much they influenced the training*** of that neuron.

What can we make of this?

There is no obvious relationship between the neurons and the structure of the training data here.



Example: Simple Classification

The second layer neurons, using the inputs from the first layer, learn the final model.

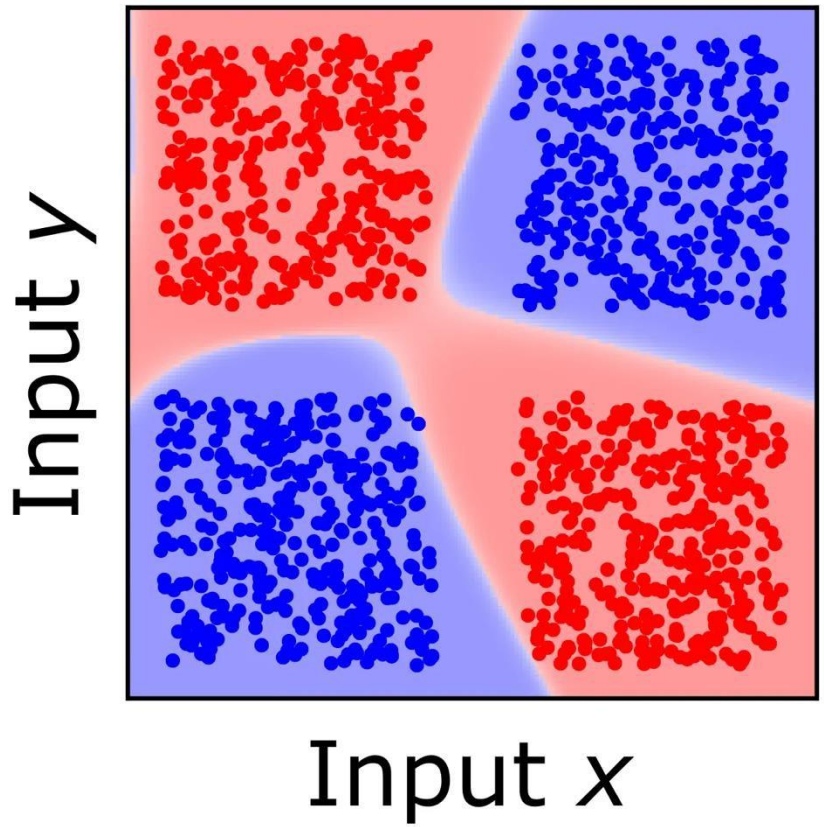
However, the first and second layer are learnt simultaneously – **at the same time**.

"Each neuron does a little bit of everything in order to handle the mess it receives from the previous layer," in Blazek's words.

More formally, this is called a ***distributed representation.***

Therefore, **each neuron does not have a clear function that reflects the structure of the task** it is trying to learn.

This leads to the black box; each neuron forms seemingly nonsensical decision boundaries.



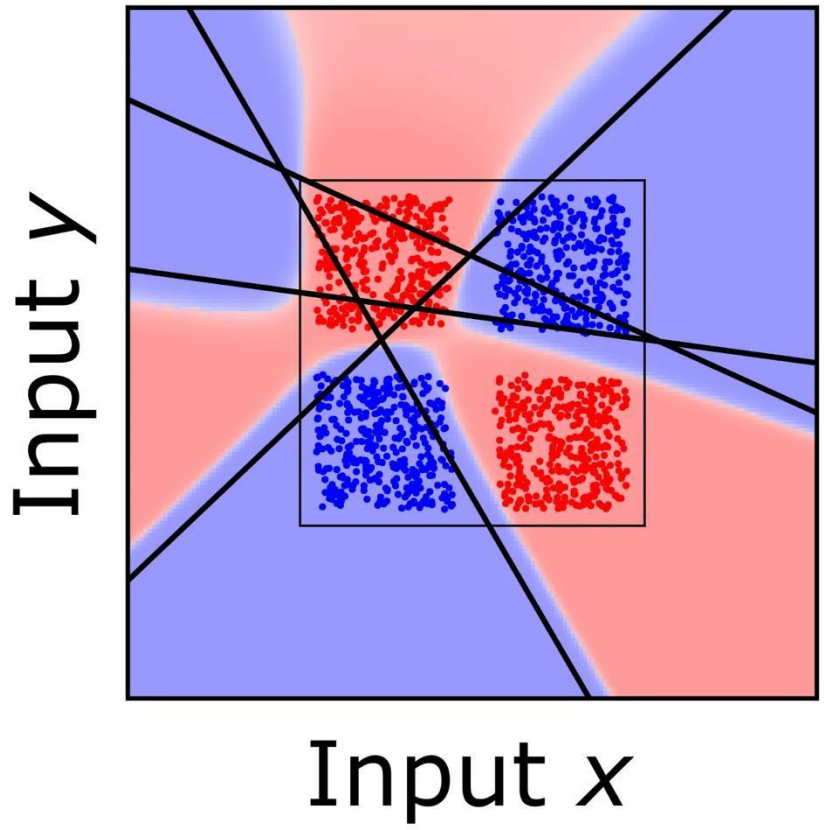
Based on an example by Paul J. Blazek;

Example: Simple Classification

So, what happens when we generalize it outside of the training data space?

Here we see the first layer boundaries and the final model.

As a result, we have created unexpected, undesirable behaviors outside the domain of the training samples.



And we may not have known that,

Or have a good understanding of why.

Based on an example by Paul J. Blazek:
https://towardsdatascience.com/why-we-will-never-open-deep-learnings-black-box-4c27cd335118

Importantly, this shows us that the “Black Box Problem” is related to *how* complexity and learning can emerge in unintuitive ways (to us humans) NOT from neural network’s outputs being “unexplainable”.

Complexity and Emergence

We asked…

- *Why do we tend to consider neural networks to be black boxes?*
- *Is it just really difficult mathematics?*
- *Or is there something that makes them intrinsically unexplainable?*

And it's because the complex interactions between many components can be difficult or even impossible to predict.

In this case, we don't know how all the individual neurons work together to arrive at the final output. A lot of times it isn't even clear what any particular neuron is doing on its own.

Now consider that many modern neural networks have millions or even billions of neurons. (E.g., GPT-3 has 185 billion weights; DALL-E has 12 billion.)

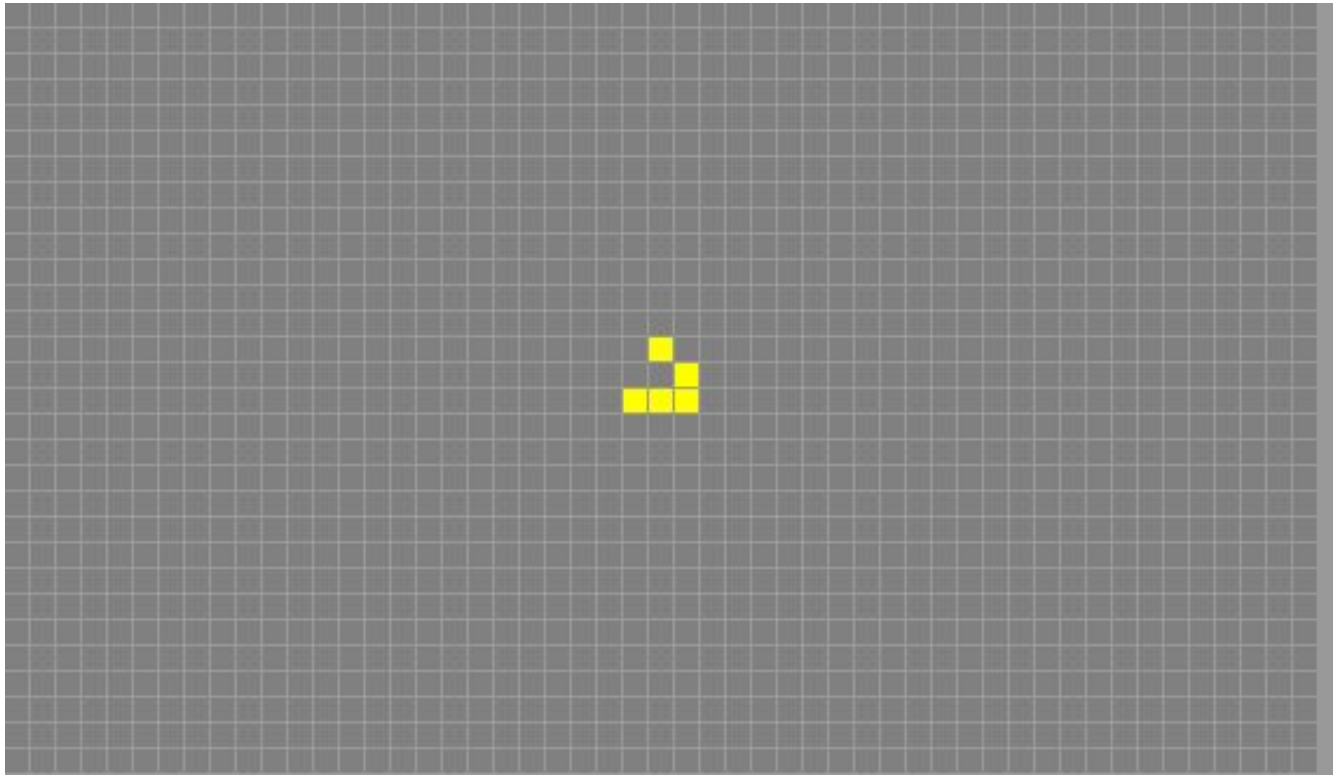
The phenomenon of many small interactions giving rise to difficult / impossible to predict

A Different Example of Emergence: Conway's Game of Life

A 2D 'Cellular Automata' invented in 1970 by mathematician John Conway.

Basically, a very rudimentary model of life on a 2D grid.

Each cell is either populated ('alive') or unpopulated ('dead').



A Different Example of Emergence: Conway's Game of Life

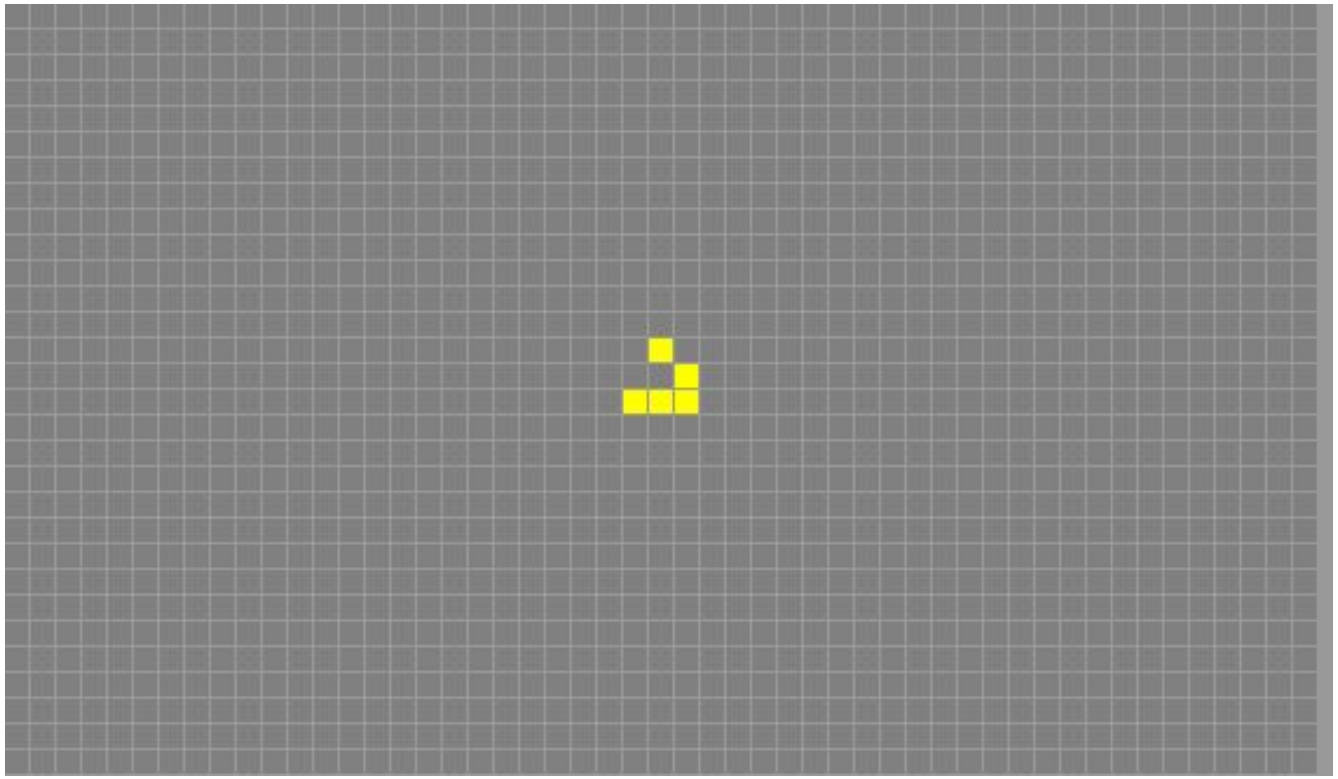
The Rules:

For a cell that is populated:

- Each cell with 1 or no neighbours dies, as if by loneliness.
- Each cell with 4 or more neighbours dies, as if by overpopulation.
- Each cell with 2 or 3 neighbours survives.

For a cell that is unpopulated:

- Each cell with 3 neighbours becomes populated.



Can you predict what will happen when we apply these rules in a loop on this grid?

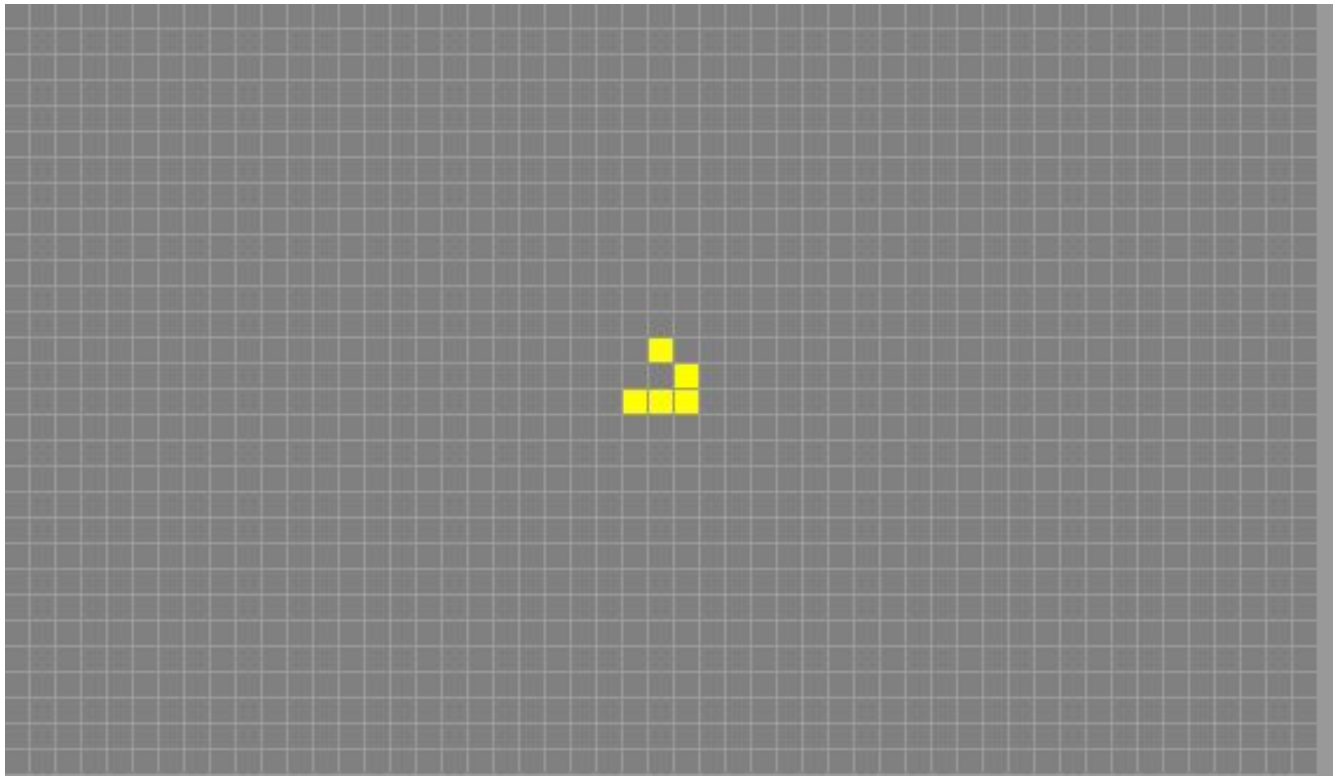
A Different Example of Emergence: Conway's Game of Life

Try it out!

https://www.playgameoflife.com/

See if you can find any way to predict what behaviour you will see, when these simple rules interact in complex ways. Can you even predict simply if it will stabilize or die out?

This example is included here not because it has anything to do with Neural Networks (it doesn't), but because it is a great way to build intuition about the challenges of emergence.



This configuration is the 'glider', which has been adopted as a symbol of hacker culture, e.g.,
http://www.catb.org/hacker-emblem/

Conclusion

By the end of this lecture, you should be able to:

- Explain the idea of linear separability in classification problems,
  and why multiple layers of neurons are needed for non-linearly separable problems.
- Explain why training multiple layers of a Neural Network together can lead to neural networks that are difficult to interpret, and the presence of non-intuitive behaviour at the level of individual neurons.
- Explain why this creates uncertainty around generalization beyond the training set.
- Appreciate and discuss the concept of emergence and how it can apply to AI systems.
- Relate these concepts to the overall questions of trustworthiness of AI systems.